Midas User Manual

v0.1.0 - 21b648e1

Contents

List of Listings

List of Tables

I. Introduction

Python is a very popular programming language, especially in data sciences. However, it has been designed for simplicity, distancing itself from typed languages such as Java or C to embrace dynamic typing. What this means is that in Python, type checks are deferred to runtime when operations are concretely executed.

For developers, it might seem like a great way of simplifying the language and making it very flexible, but it does come with a cost. Indeed, type errors are very easy to make in Python. While passing an integer where a string is expected might not be an issue in some cases, these are the sort of thing that can cause crashes or incorrect results without a clear diagnostic to help the user fix it.

Fortunately, developers using IDEs or properly configured text editors can benefit from external type checkers such as MyPy which will perform static type analysis of their Python code. Some can also be configured to be very strict, forcing the user to make the whole code typeable statically, thus avoiding any runtime type errors.

This is not the end of the problem though. Some parts of a program, especially in data related fields, may not be available at “compile-time”. For example, a dataset can be loaded from an external file, or data can be fetched from an API, with no guarantees of having the expected format when analyzing the code statically.

In turn, that can cause a range of loud and silent errors at runtime. A malformed number will probably crash the program when trying to convert it, but a NaN in a series of value might just produce wrong results without any exception. Combine this with often long-running data-processing pipelines and this is how developers can waste hours of precious computation time.

Midas is a type system which can be used on top of Python to provide better type checking capabilities and gradual typing. It aims at providing optional but strict type annotations and casting operations which can produce runtime assertions. It also allows the user to define dependent types with value constraints that are translated into runtime checks.

II. Installation

Midas comes as a very light Python package that you can install on your system in a few simple steps.

II.1. Requirements

Here below are the requirements for installing Midas. All Python dependencies will be installed by `uv` in the installation process described in Section II.2.

- Python 3.11+
- `uv`

II.2. Steps

1. Clone the repository

```bash
git clone https://git.kb28.ch/HEL/midas.git
```

2. Navigate inside the directory

```bash
cd midas
```

3. Install Midas as a tool in your local user space

```bash
uv tool install .
```

And that's it ! You can now use Midas commands anywhere, like this:

```bash
midas --help
```

III. Quick Start

This chapter will give you the keys to quickly start using Midas in your project.

III.1. Defining custom types

To begin with, you might want to define some custom types for your project, to avoid handling anonymous float values everywhere. To do so, create a `*.midas` file in your project, and write some definitions for your types. See Section IV for more information on syntax and features.

Listing 1 shows a simple example of what it might look like.

types.midas — Midas

```
type Meter = float
extend Meter {
  def __add__: fn(Meter, /) -> Meter
  def __sub__: fn(Meter, /) -> Meter
}

type Coordinate = object
extend Coordinate {
  prop x: Meter
  prop y: Meter
}
```

Listing 1: Example Midas type definitions

You can check for any syntax error using the following command:

Bash

```
midas validate types.midas
```

When you are happy with your definitions, you can generate Python stubs to use in your source code. This allows other type checkers like MyPy to recognize your custom types and avoid reporting them as undefined. It can also help catch some type errors in your IDE.

Bash

```
midas stubs types.midas -o stubs.pyi
```

This command will generate a file as shown in Listing 2, providing stub classes to represent the type lattice including methods and properties.

stubs.pyi — Python

```
from __future__ import annotations

class Meter(float):
    def __add__(self, _0: Meter, /) -> Meter: ...
    def __sub__(self, _0: Meter, /) -> Meter: ...

class Coordinate(object):
    x: Meter
    y: Meter
```

Listing 2: Generated stubs from example definitions of Listing 1

III.2. Using Midas in Python

You can now write your Python program as you would normally. You can import your custom types from the generated stubs file and use them in type annotations.

You can also import the `cast` and `unsafe_cast` functions from `midas.typing` to explicitly cast a value to a specific type (see Section V.7 for more information).

An example Python script is shown in Listing 3, demonstrating how you can use custom types in type annotations. Notice the comments describing errors that will be caught by the type checker in Section III.3.

script.py Python

```python
from lib import load_coordinate
from midas.typing import cast
from stubs import Coordinate, Meter

p1 = cast(Coordinate, load_coordinate(0))
p2 = cast(Coordinate, load_coordinate(1))

diff_x = p2.x - p1.x
diff_y = p2.y - p1.y

dist = diff_x + diff_y

p2.x += cast(Meter, 1)
p2.y = True  # invalid, wrong type
p2.z = 3  # invalid, no property 'z' on Coordinate
p2.x.a = 3  # invalid, no properties on Meter
```

Listing 3: Example Python script

III.3. Type checking

Now that you have defined some types and written a script, you can run the type checker with the following command. You can also skip this step and directly run the compilation command in Section III.4.

```bash
midas check -t types.midas script.py
```

Bash

III.4. Compiling

The final step is to compile your code. This step will produce a runnable Python script, including runtime assertions generated by `cast` expressions.

```bash
midas compile -t types.midas script.py
python3 build/midas/script.py
```

Bash

IV. Midas Language Reference

In this chapter, you will find a complete reference for the Midas definition language.

A `*.midas` file contains a number of statements, which can be:

- **`alias`** statements (see Section IV.1): to define a new type alias
- **`type`** statements (see Section IV.2): to define a new type
- **`extend`** statements (see Section IV.3): to define member of a type
- **`predicate`** statements (see Section IV.4): to define named predicates that can be used in constraint types

IV.1. Alias Statement

An **`alias`** statement lets you define a new type alias. It requires a unique name and base type.

While a `type` statement (see Section IV.2) allows generic definitions, aliases are purely a for givin an alternative name to a type.

```midas
alias MyType = float
```

Listing 4: Simple `alias` statement declaring a new type "`MyType`" equivalent to `float`

This statement defines a new type called `MyType` which is equivalent to `float`. `MyType` and `float` can be used interchangeably.

IV.2. Type Statement

A **`type`** statement lets you define a new type. It requires a unique name and base type.

The simplest form of a **`type`** statement is:

```midas
type MyType = float
```

Listing 5: Simple `type` statement declaring a new type "`MyType`" as a subtype of `float`

This statement defines a new type called `MyType` which is a subtype of `float`. `MyType` is a `float` but a `float` is not necessarily `MyType`.

IV.2.1. Builtin / base types

A number of base types are provided out of the box, which can be used to derive other types.

They correspond to Python's builtin types: `object`, `str`, `float`, `int`, `bool`, `list`, `dict`, `None`.

Some differences are to be noted however.

1. `bool` is not a subtype of `int`
2. `list` are homogeneous, i.e. all items must be of the same type
3. `dict` keys and values are homogeneous, i.e. all keys must be of the same type and all values must be of the same type (can be different from keys).

IV.2.2. Function types

A function type is written in a similar notation to Python function definitions:

```midas
type Repeater = fn(text: str, count: int) -> str
```

Listing 6: Simple function type definition

Midas supports positional-only, keyword-only and mixed arguments (using the `/` and `*` separators). You may omit the name of positional-only arguments. The return type is required.

Optional parameters can be indicated by adding a question mark (`?`) after their type:

```
type Repeater = fn(text: str, count: int, *, sep: str?) -> str
```
Midas

Listing 7: Function type definition with an optional keyword-only parameter

> ⚠ **Warning**
>
> Sink arguments (`*args`, `**kwargs`) are not currently supported.

IV.2.3. Constraint types

A useful feature provided by Midas is the possibility to combine types with custom value constraints. For example, you might want to define a type for positive amounts of money:

```
type Money = float
type Income = Money where _ >= 0
```
Midas

Listing 8: Simple constraint type definition

Constraints can be combined with any type using the `where` keyword, followed by a constraint expression (see Section IV.5).

IV.2.4. Generic types

For more complex types, you might want to use type parameters. For example, to define a container, we might write:

```
type Container[T] = object
```
Midas

Listing 9: Simple generic container type definition

To better refine a generic type, you can also bound type parameters using the following syntax:

```
type Container[T <: float] = object
```
Midas

Listing 10: Generic container type definition with a bound

This can be read as "`Container` is a generic type which takes one type parameter `T` that must be a subtype of `float`".
You can use a generic type, i.e. instantiate it, by using a similar syntax with concrete type as arguments:

```
type MyContainer = Container[MyType]
```
Midas

Listing 11: Application of a generic type

Generic types can also take multiple parameters, which are then separated by commas:

```
type ZipCodeRegistry = dict[int, str]
```
Midas

Listing 12: Application of a multi-parameter generic type

The *body* of a generic type, i.e. the right-hand side of the definition, can contain or even be equal to any number of its parameters.[1] For example, the following is a valid type statement:

```
type Price[T <: Currency] = T where _ > 0
```
Midas

Listing 13: Type parameters in a generic type's body

[1] The latter is not something that is expressible in standard Python, yet it brings a semantic distinction on top of structurally equivalent values.

IV.2.5. `Column` / `Frame` types

To provide useful type-checking for data engineers, Midas offers two special types: `Column` and `Frame`. Their goal is to help type check Pandas' `Series` and `DataFrame` respectively.

IV.2.5.1. `Column`

The `Column` type is a generic type used to represent a `pandas.Series` object. You can use it like any other generic type and it will provide type checking for some common methods and attributes offered by Pandas.

```
1 type Temperature = float                                    Midas
2 alias Temperatures = Column[Temperature]
```

Listing 14: Simple column type definition

IV.2.5.2. `Frame`

The `Frame` type is a super-powered generic type used to represent a `pandas.DataFrame` object. In place of type arguments, `Frame` accepts a schema, i.e. a series of column definitions. Listing 15 show how you can define a simple frame type with 3 columns:

- `name`: a column of `Name` values
- `age`: a column of `int` values
- `height`: a column of `float where _ >= 0` values

Notice that you don't need to specify `Column` types.

```
1 type Name = str where len(_) != 0                           Midas
2 alias Data = Frame[
3   name: Name,
4   age: int,
5   height: float where _ >= 0
6 ]
```

Listing 15: Simple frame type definition

IV.3. Extend Statement

Type statements allow you to define new types, kind of like type aliases. However, a type might have properties or methods of its own. These might override those of the parent type or be brand new members.

This is where the `extend` statement comes into play. It allows defining members on a given type. Members can either be properties (`prop`) or methods (`def`). The only difference between the two is that methods must be functions and can be overloaded.

Here is a simple example showing how to define a property and a method on a custom type:

```
type MyType = float
extend MyType {
  prop norm: float
  def double: fn() -> MyType
}
```

Midas

Listing 16: Simple `extend` statement defining a property and a method

An `extend` statement can appear anywhere after the type it extends has been defined. You may want to override Python's dunder methods to implement type checking for some basic operators, like `__add__` for the `+` operator.

```
type Money = float
extend Money {
  def __add__(Money, /) -> Money
  def __mul__(float, /) -> Money
}
```

Midas

Listing 17: Simple `extend` statement overriding some dunder methods

When extending generic type, you must specify the whole type, including its parameter(s):

```
type Container[T <: float] = object
extend Container[T <: float] {
  prop content: T
  def set_content: fn(content: T) -> None
}
```

Midas

Listing 18: Generic `extend` statement using type parameters in the declared members

IV.4. Predicate Statement

A **`predicate`** statement lets you define a named constraint expression, like a function, which can then be used in other constraint expressions (either in other predicate statements or in constraint types). See Section IV.5 for more information about the syntax of constraint expressions.

The left-hand side of a predicate statement is written as a function signature, without a return type. The right-hand side is a constraint expression. For example:

```
1 predicate is_positive(v: float) = v >= 0
```
Midas

Listing 19: Simple `predicate` statement defining an `is_positive` predicate

The left-hand side can also be curried to allow partial application. For example:

```
1 predicate in_range(mn: float, mx: float)(v: float) = mn <= v & v <= mx
2 predicate is_ratio = in_range(0.0, 1.0)
```
Midas

Listing 20: Curried `predicate` statement and partial application

Notice that the second predicate statement doesn't take any parameters. This is simply a partial application of another predicate, kind of like an alias. You can use it in other expressions to finalize the call:

```
1 type Efficiency = float where is_ratio(_)
```
Midas

Listing 21: Constraint type definition using the partially applied predicate from Listing 20

Of course you can also directly call `in_range`:

```
1 type Efficiency = float where in_range(0.0, 1.0)(_)
```
Midas

Listing 22: Full call of curried predicate from Listing 20

When compiled, named predicates are translated to Python functions which are used in runtime assertions. Only predicates that are referenced are compiled.

IV.5. Constraint Expressions

Constraint expressions are Python-like expressions which can appear in **`predicate`** statements or in constraint types.

They can contain comparisons, simple computations, logical operations and must evaluate to a boolean value.

Context is quite restricted inside these expressions. You can only reference some builtin functions, such as type constructors (`float(...)`, `str(...)`, etc.), parameters of predicate statements, and named predicates. In constraint type, the special variable _ can be used to reference the value targeted by the type. For example:

```
1 predicate not_nan(v: float) = v != float("nan")                    Midas
2 type RealFloat = float where not_nan(_)
```

Listing 23: Example constraint expressions

In the predicate statement (Listing 23-1), we reference the parameter `v` and the builtin `float` function. In the constraint type definition (Listing 23-2), we then reference the named predicate `not_nan`, passing the value targeted by the type itself (_)

V. Supported Python Syntax

Midas integrates naturally in Python via type annotations. Through generated stubs, even other type checker can detect your custom types (see Section VI.7).

It has been designed to leave the user free of typing any amount of their code but be strict about the parts that are annotated. By default, any untyped Python expression is assigned `UnknownType`.

Any operation is permitted on `UnknownType` and will result in `UnknownType` values.

The moment an expression can be typed, that be thanks to an annotation or a literal value, the type checker kicks in and will validate your statements.

Because Python is very flexible language with many features, some expressions and statements might be more complex to properly type check, thus only a subset of the Python language is fully supported. This chapter lists all supported features of Python and how they affect type checking.

Some examples are presented in the following sections in the form of code blocks. Highlights in the code blocks indicate the type assigned to each expression by the type checker. Some types may be omitted for readability. For example:

```python
1 v = 3 int
2 print(v int)
```

V.1. Literals

Literal Python values are type checked using builtin types. Lists and dictionaries of literals are also typed liked literals. This does not include comprehension lists/dicts (`[. for . in .]`), nor formatted strings (`f"..."`). Table 1 shows the list of supported literal values and their type.

Example value	Judged Type
`42`	`int`
`3.14`	`float`
`True`	`bool`
`"Midas"`	`str`
`None`	`None`
`[1, 2, 3]`	`list[int]`
`{1: "One", 2: "Two"}`	`dict[int, str]`
`("1", 1, True)`	`tuple[str, int, bool]`

Table 1: Supported literal values and their judged types

V.2. Assignments

Variable assignments allow assigning a new value to a variable. For the type checker, this implies two things:

1. If the variable was not already declared in the current scope, it is declared at that point with the type of the right-hand side expression
2. If the variable was already declared, the type of the right-hand side expression is checked against the declared type of the variable. Only a subtype of the variable's type can be assigned to it

Once a variable has been given a type, it cannot be changed in the same scope.

The walrus operator (`:=`) is not currently supported.

A simple annotation declaration, without assigning a value, is enough to declare a variable. For example:

```python
var: float
```

Listing 24: Bare Python variable annotation without assignment

Because unpacking is not supported, assigning to multiple values is also not handled by the type checker. For more information about type annotations, see Section V.8

V.3. Arithmetic

- All basic binary operators are supported, through dunder methods.
- All comparison operators except `in` are supported.
- All unary operators are supported (+, -, ~).
- All logical operators are supported (`and`, `or`, `not`).

V.4. Ternary operator

The ternary operator `. if . else .` is supported. As for `if` statements (see Section V.5.1), the test expression must be a boolean. Additionally, both branches must be of the same type.

For example:

```python
parity = ("even" str if num % 2 == 0 bool else "odd" str) str
```

Listing 25: Typing of ternary operator

V.5. Control flow

Some control flow features are supported. For the limited code of this project, not all constructs are supported. The following are those currently handled and type checked by Midas.

V.5.1. `if` / `elif` / `else`

Conditional statements are checked relatively strictly by Midas. The test expression, i.e. what comes after the `if` keyword, must be a boolean. While Python allows introducing and leaking new variables from inside an `if` statement, Midas will strictly forbid leaks by restraining bindings to the scope they are defined in. For example, the following Python code will not compile with Midas:

```python
age = 22
if age >= 18:
    msg = "You're an adult"
else:
    msg = "You're still a child"
print(msg)  # -> unknown variable 'msg'
```

Listing 26: `if`/`else` statement cannot leak variables

V.5.2. `for` loops

Simple forms of `for` loops can be used, that is using a single variable and iterating over an object implementing the `__getitem__` method. Like above in Section V.5.1, leaking variables from inside the loop is ignored.

`for-else` statements are not supported. `while` loops are also not supported.

V.6. Functions

You can define functions as usual and the type checker will do its best to type it. Apart from argument sinks (`*args`, `**kwargs`), all forms of parameter specifications are supported (positional-only, keyword-only, mixed, optional).

As for the rest of your code, type annotations are optional, but recommended. If you omit the return type hint, the type checker will try to infer it from the function body and its return statements. If you did specify a return type, all return paths must return values that are subtypes of the type hint.

```python
def double(value: float) -> float:
    return value float * 2 float
result = double (value: float) -> float (4.0 float) float
```

Listing 27: Typing of function's body and call

Anonymous functions (`lambda`) are not yet supported

V.7. Casts

> **Info**
>
> The functions discussed in this section are provided by the `midas.typing` submodule. You can import them in your script like so:
>
> ```python
> from midas.typing import cast, unsafe_cast
> ```
>
> Listing 28: Importing cast functions

Sometimes, you may want to use a value whose type is not known to the type checker in a place where it expects a particular type. In that case, if you do know that the runtime type will correspond to what is expected, you can use a `cast` expression.

Similar to the `cast` function from the `typing` package of Python's Standard Library, it allows telling the type checker that a value has a given type. While `typing`'s function doesn't have any runtime side-effect, Midas' will generate runtime assertions, ensuring that your statement is true when running the code. What cannot be checked statically is checked at runtime.

In the following example, a runtime check would be generated to ensure that the value is indeed a `float` and that it satisfies the type's constraint (i.e. `>= 0`):

```python
typed_value = cast(PositiveFloat, unknown_value UnknownType)
print(typed_value PositiveFloat)
```

Listing 29: Typing of `cast` expression

> **Warning**
>
> Assertions are statements inserted just before a statement using a `cast` expression. This means that the expression is evaluated *before* its actual intended usage location, which might cause issues if you rely on logical operator short-circuiting. See Section VII.1 for more information.

There may be some cases where the cost of checking a value at runtime is simply not worth the safety, for example when dealing with a big dataset. If do wish so, you can use `unsafe_cast` which will only tell the type checker the type of the value, without generating a runtime assertion. This maps to the default behavior of `typing`'s own `cast` function.

If the value passed to `cast` or `unsafe_cast` is a literal (e.g. an integer, a string, a list of literals, etc.), the assertion is evaluated *at compile-time* and no runtime assertion is generated.

V.8. Annotations / Type Hints

Vanilla Python already lets you use type hints to specify the type of variables and function parameters.

Midas use them to type check your code. Additionally, it allows you to use a special syntax to define a `Frame` type directly in these annotations.

Because these annotations are not interpretable by Python, your integrated type checker might complain loudly about them being invalid. A workaround is to silence it by adding a type comment at the end of the line, as shown in Listing 30.

```python
var: Frame[name: str, age: float]  # type: ignore # noqa: F821
```

Listing 30: MyPy's and Pylance's complaints about custom type annotation can be silenced with type comments

V.8.1. Frame type annotation

The syntax is similar to how you can define frame types in the Midas language (see Section IV.2.5.2). The only difference is that types can only be name references; you cannot inline constraint types.

The example of Listing 31 shows how you can annotate a dataframe with some columns directly in Python.

```python
df: Frame[name: Name, age: float, height: Length[Meter]] = ...
```

Listing 31: Frame type annotation in Python

VI. Commands

Midas offers several commands to parse, check and compile your code.

All commands presented in this chapter are subcommands of the `midas` command. For example:

```shell
midas compile script.py -t types.midas
```

Listing 32: Example midas command

VI.1. Type Checking (`check`)

```shell
midas check -t types.midas source.py
```

Listing 33: Example usage of `midas check`

This command parses the given files and run the type checkers against the Midas definitions and Python program. Diagnostics are then printed showing warnings and errors.

VI.1.1. Parameters

- `FILE`: the Python file to type check

VI.1.2. Options

- `-t` / `--types` `FILENAME`: Midas files defining type definitions. This option can be used multiple times. Files should end with `.midas`
- `-l` / `--highlight` `FILENAME`: if set, a highlighted version of the Python file showing inline diagnostics will be written to the given file

VI.2. Compiling (`compile`)

```shell
midas compile -t types.midas source.py
```

Listing 34: Example usage of `midas compile`

With the `compile` command, you can process a source Python file, with any number of custom type definition files, and the type checker will verify the coherence of your program and generate the runnable code with valid syntax and runtime assertions.

VI.2.1. Parameters

- `FILE`: the Python file to compile

VI.2.2. Options

- `-t` / `--types` `FILENAME`: Midas files defining type definitions. This option can be used multiple times. Files should end with `.midas`
- `-s` / `--stubs` `TEXT`: the file name of generated stub files. Each instance of this option maps to an instance of `-t`. For example:

  ```shell
  midas -t types1.midas -s stubs1 -t types2.midas -s misc source.py
  ```

  will compile to the following files:

  ```
  └─ build/
     └─ midas/
        ├─ stubs1.py
        ├─ misc.py
        └─ source.py
  ```

- `--ignore-errors`: if set, error diagnostics will not prevent the generator from running

VI.3. Formatting (`format`)

```shell
midas format types.midas
midas format types.midas -o formatted.midas
```

Listing 35: Example usage of `midas format`

This command parses the given Midas file and outputs a pretty printed file from the AST.

VI.3.1. Parameters

- `FILE`: the Midas file to format

VI.3.2. Options

- `-o` / `--output FILENAME`: if set, the output is written to the given file instead of STDOUT

VI.4. Highlighting (`highlight`)

```shell
midas highlight source.py
midas highlight source.py -o highlighted.html
midas highlight types.midas
midas highlight types.midas -o highlighted.html
```

Listing 36: Example usage of `midas highlight`

The `highlight` command takes in a source file (Python or Midas), runs the appropriate parser and outputs an HTML file containing the source code with added highlighting. This highlighting takes the form of hoverable annotations showing some of the parsed structures (e.g. a function definition, an assignment, a generic type, etc.)

VI.4.1. Parameters

- `FILE`: the Python or Midas file to highlight

VI.4.2. Options

- `-o` / `--output FILENAME`: if set, the output is written to the given file instead of STDOUT

VI.5. Dumping the AST (`parse`)

```shell
midas parse source.py
midas parse types.midas
```

Listing 37: Example usage of `midas parse`

For debugging purposes, you can use this command to output the AST parsed from a Python or Midas file

VI.5.1. Parameters

- `FILE`: the Python or Midas file to parse

VI.5.2. Options

- `--raw`: if `FILE` is a Python file, the raw AST is returned, as produced by the builtin `ast` module, instead of the custom AST nodes used by Midas internally. This flag has no effect on Midas files

VI.6. Dumping the Registry (`dump-registry`)

```shell
midas dump-registry -t types.midas
```

Listing 38: Example usage of `midas dump-registry`

This command processes the given Midas definitions and dumps the contents of the types registry.

VI.6.1. Options

- `-t` / `--types` `FILENAME`: Midas files defining type definitions. This option can be used multiple times. Files should end with `.midas`

VI.7. Generating Stubs (`stubs`)

```shell
midas stubs types.midas
```

Listing 39: Example usage of `midas stubs`

This command generates Python stubs from a Midas definition file

VI.7.1. Parameters

- `FILE`: the source Midas file

VI.7.2. Options

- `-o` / `--output` `FILENAME`: if set, the stubs are written to the given file instead of the default `FILE.pyi`
- `-w` / `--watch`: if set, the input file will be watched and any changes to it will regenerate the stubs, otherwise the command exits after generating the stubs once

VI.8. Showing Type Judgements (`types`)

```shell
midas types -t types.midas source.py
```

Listing 40: Example usage of `midas types`

This command type checks the given Python source file and logs all typing judgements made by the type checker.

VI.8.1. Parameters

- `FILE`: the Python file to type check

VI.8.2. Options

- `-t` / `--types` `FILENAME`: Midas files defining type definitions. This option can be used multiple times. Files should end with `.midas`
- `-l` / `--highlight` `FILENAME`: if set, a highlighted version of the Python file showing inline diagnostics will be written to the given file

VI.9. Validating Definitions (`validate`)

```shell
midas validate types.midas
```

Listing 41: Example usage of `midas validate`

This command lets you validate a Midas definition file by running the parser and type checker, verifying syntax and references.

VI.9.1. Parameters

- `FILE`: the Midas file to validate

VI.9.2. Options

- `-l` / `--highlight` `FILENAME`: if set, a highlighted version of the Midas file showing inline diagnostics will be written to the given file

VII. Known limitations

VII.1. Eager evaluation in runtime assertions

The process of generating assertions to ensure safety at runtime, mainly for `cast` expressions, leads to the creation of aliases for the expressions being casted. These alias definitions eagerly evaluate before the assertion, and most importantly before the real usage location. This means that you should avoid using `cast` expressions inside logical expressions like `and` or `or`, because the normal "short-circuit" behavior will be irrelevant to the evaluations of the operands.

For example:

```python
def foo():
    print("Foo")
    return True
def bar():
    print("Bar")
    return True
result = foo() or bar()
# Foo
# Bar
```

Listing 42: Runtime assertions may eagerly evaluate expressions and bypass logical operator's short-circuit